



**TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN**

ĐỒ ÁN CƠ SỞ DỮ LIỆU PHÂN TÁN

ĐỀ TÀI: ỨNG DỤNG GỌI XE THEO VỊ TRÍ

Giảng viên hướng dẫn: **TS.VŨ NGỌC THANH SANG**

Sinh viên thực hiện:

HỒ NGỌC PHƯƠNG NHI	3123411210
NGUYỄN HOÀNG TRÍ	3123411311
LÂM TRÍ VĨNH	3123411343
PHẠM THIÊN ÂN	3123411022

Lớp: **DCT123C2**

Thành phố Hồ Chí Minh, tháng 05 năm 2026

TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN

ĐỒ ÁN CƠ SỞ DỮ LIỆU PHÂN TÁN

Thành phố Hồ Chí Minh, tháng 05 năm 2026

LỜI MỞ ĐẦU

Trong thời đại công nghệ số hiện nay, các ứng dụng thời gian thực như ứng dụng gọi xe đang đóng vai trò thiết yếu trong đời sống hàng ngày của hàng triệu người. Một trong những yếu tố mang tính sống còn quyết định sự thành công của các ứng dụng này chính là trải nghiệm người dùng, vốn chịu ảnh hưởng rất lớn bởi độ trễ của mạng. Chẳng hạn, trong một kiến trúc tập trung truyền thống, nếu một người dùng tại TP.HCM phải kết nối đến một máy chủ duy nhất đặt tại Hà Nội, họ chắc chắn sẽ phải đối mặt với tình trạng độ trễ cao, làm giảm tính hiệu quả và sự hài lòng khi sử dụng dịch vụ.

Để giải quyết triệt để bài toán này, việc chuyển đổi sang kiến trúc hệ thống Cơ sở dữ liệu phân tán là một xu hướng tất yếu. Hệ thống phân tán không chỉ giúp tối ưu hóa hiệu suất bằng cách đưa dữ liệu đến gần người dùng hơn, mà còn là lời giải cho các thách thức phức tạp trong thực tế như khả năng chịu lỗi (fault tolerance), kiểm soát đồng thời (concurrency control) và đảm bảo tính sẵn sàng cao (high availability).

Nhận thức được tầm quan trọng và tính ứng dụng thực tiễn của kiến trúc này, nhóm chúng em đã quyết định chọn và thực hiện Đề tài 4: Ứng dụng Gọi xe theo Vị trí. Mục tiêu cốt lõi của đề án là xây dựng một hệ thống có khả năng tự động định tuyến dựa trên vị trí, phân chia dữ liệu rõ ràng với Server Miền Bắc phục vụ khu vực Hà Nội và lân cận, trong khi Server Miền Nam phục vụ khu vực TP.HCM và các vùng lân cận.

Bên cạnh đó, đề án tập trung sâu vào việc cấu hình nhân bản (Master-Slave Replication) và xây dựng cơ chế Failover chuyển đổi dự phòng. Điểm nhấn kỹ thuật của giải pháp là đảm bảo tính liên tục của dịch vụ: ngay cả khi máy chủ chính (Primary) gặp sự cố sập nguồn, người dùng vẫn có thể kết nối vào máy chủ dự phòng (Replica) dưới chế độ Read-Only để đăng nhập và xem lịch sử chuyến đi, dù không thể đặt chuyến mới.

Thông qua đề án này, nhóm kỳ vọng không chỉ hiện thực hóa được các lý thuyết đã học mà còn tích lũy thêm kinh nghiệm thiết kế kiến trúc hệ thống chịu tải và xử lý sự cố trong môi trường phân tán.

LỜI CẢM ƠN

Để hoàn thành đồ án môn học Cơ sở dữ liệu phân tán này, chúng em đã nhận được sự hỗ trợ, hướng vô cùng to lớn từ thầy.

Trước hết, chúng em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến Giảng viên phụ trách bộ môn Cơ sở dữ liệu phân tán. Trong suốt quá trình học tập, Thầy không chỉ truyền đạt những kiến thức nền tảng vô giá về thiết kế và quản trị cơ sở dữ liệu mà còn tận tình định hướng, hướng dẫn nhóm vượt qua những khó khăn về mặt kỹ thuật trong quá trình cài đặt cơ chế đồng bộ dữ liệu, phân mảnh và xử lý lỗi của hệ thống. Những lời khuyên và nhận xét khắt khe nhưng đầy tâm huyết của Thầy chính là kim chỉ nam giúp chúng em hoàn thiện sản phẩm này.

Mặc dù nhóm đã nỗ lực hết sức để hoàn thiện hệ thống và báo cáo kỹ thuật, nhưng do những giới hạn về mặt thời gian cũng như kinh nghiệm thực tiễn, sản phẩm chắc chắn không thể tránh khỏi những thiếu sót nhất định. Chúng em rất mong nhận được sự thông cảm, cũng như những ý kiến đánh giá, góp ý quý báu từ Thầy để đồ án được hoàn thiện hơn và nhóm có thêm hành trang kiến thức vững chắc cho con đường phát triển nghề nghiệp sau này.

Chúng em xin chân thành cảm ơn!

MỤC LỤC

CHƯƠNG 1: TỔNG QUAN ĐỒ ÁN	1
1.1 Bối cảnh và vấn đề thực tế.....	1
1.2 Mục tiêu đồ án	1
1.3 Phạm vi đồ án	2
1.4 Công nghệ sử dụng	2
CHƯƠNG 2: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG	4
2.1 Kiến trúc tổng thể	4
2.2 Thiết kế cơ sở dữ liệu	5
2.2.1 Các bảng chính	5
2.2.2 Chiến lược phân vùng dữ liệu	5
2.3 Thiết kế Query Router (DatabaseService).....	6
2.4 Cơ chế Replication (PostgreSQL 16 Streaming).....	7
CHƯƠNG 3: TRIỂN KHAI HỆ THỐNG.....	8
3.1. Cấu hình môi trường Docker	8
3.2. Cấu hình Replication	12
3.2.1. Cài đặt Primary.....	12
3.2.2. Khởi tạo Replica.....	13
3.3. Triển khai LocationRoutingMiddleware	14
3.4. Luồng Failover và Chế độ Read-Only	15
3.5. Các API Endpoint chính	16
CHƯƠNG 4: KIỂM THỬ	20
4.1. Chiến lược kiểm thử	20
4.2. Test Case chi tiết.....	20
CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	25
5.1. Hạn chế hiện tại	25
5.2. Hướng phát triển	25

CHƯƠNG 1: TỔNG QUAN ĐỒ ÁN

1.1 Bối cảnh và vấn đề thực tế

Trong các ứng dụng gọi xe thời gian thực (ride-hailing), độ trễ mạng (network latency) ảnh hưởng trực tiếp đến trải nghiệm người dùng. Người dùng tại TP. Hồ Chí Minh kết nối đến máy chủ đặt ở Hà Nội sẽ gặp độ trễ cao hơn, làm chậm đăng nhập, truy vấn chuyển và thao tác đặt xe.

Bên cạnh đó, trong môi trường phân tán, khả năng sẵn sàng cao (High Availability) là yêu cầu bắt buộc. Sự cố một node không được phép làm tê liệt toàn hệ thống – người dùng phải có thể xem dữ liệu dù ở chế độ hạn chế (Read-Only).

Vấn đề	Giải pháp áp dụng trong đồ án
Độ trễ cao do khoảng cách địa lý	Định tuyến theo vị trí (Location-based Routing) – LocationRoutingMiddleware
Single Point of Failure	Master-Slave Replication + Failover tại tầng ứng dụng
Mất khả năng đọc khi Primary sập	Tự động fallback sang Replica (chế độ Read-Only)
Đồng bộ dữ liệu giữa các node	PostgreSQL 16 Streaming Replication (WAL)

1.2 Mục tiêu đồ án

- Thiết kế và minh họa phân tách dữ liệu theo khu vực địa lý (North / South) với hai cụm PostgreSQL 16 độc lập chạy trong Docker.
- Cấu hình nhân bản dữ liệu (Primary → Standby) bằng PostgreSQL Streaming Replication.
- Triển khai cơ chế failover tại tầng ứng dụng: thao tác đọc tự động chuyển sang Replica khi Primary không khả dụng.
- Thực hiện chế độ Read-Only: trả HTTP 503 có thông báo rõ ràng khi thao tác ghi yêu cầu Primary đang sập.
- Xây dựng ứng dụng thực tế gồm .NET MAUI Client và ASP.NET Core Web API để demo toàn bộ luồng.

1.3 Phạm vi đề án

- Ứng dụng mô phỏng gọi xe với 2 vùng địa lý: Miền Bắc (Hà Nội) và Miền Nam (TP.HCM).
- 4 node PostgreSQL 16: North Primary (port 5432), North Replica (port 5433), South Primary (port 5434), South Replica (port 5435) – khởi chạy qua Docker Compose.
- Web API ASP.NET Core (RideAPI) đóng vai trò Query Router, định tuyến request theo vị trí, xử lý failover và chế độ Read-Only.
- Ứng dụng .NET MAUI (APP_GoiXetheoGPS) giao tiếp với API qua HTTP/JWT, tự động gửi vĩ độ (X-User-Latitude) để hỗ trợ định tuyến.
- Giao diện quản trị Razor MVC (prefix /admin) cho phép Admin quản lý hệ thống.

1.4 Công nghệ sử dụng

Thành phần	Công nghệ	Mục đích
Mobile Client	.NET MAUI (.NET 10)	Ứng dụng gọi xe (APP_GoiXetheoGPS) – đa nền tảng
Web API / Query Router	ASP.NET Core (.NET 10)	REST API + Razor Admin + Swagger (Development)
Database	PostgreSQL 16	Primary và Replica node cho 2 vùng (Docker)
DB Access	Npgsql + Dapper	Truy cập PostgreSQL hiệu năng cao, kiểu mạnh
Replication	PostgreSQL Streaming Replication (WAL)	Đồng bộ dữ liệu Primary → Standby liên tục
Containerization	Docker Compose	Triển khai 4 node DB: docker-compose.north.yml / south.yml
Retry / Resilience	Polly	Retry tự động cho lỗi transient của Npgsql / timeout
Xác thực	JWT Bearer	Bảo vệ API; Admin dùng thêm cookie admin_jwt

Mật khẩu	BCrypt.Net-Next	Hash/verify mật khẩu người dùng
Geocoding / Bản đồ	Mapbox Geocoding API	Tìm kiếm địa điểm trong HomeMapPage (MAUI)
UI MAUI	CommunityToolkit.Maui + MVVM + Syncfusion	Điều khiển UI, MVVM helpers, theme picker

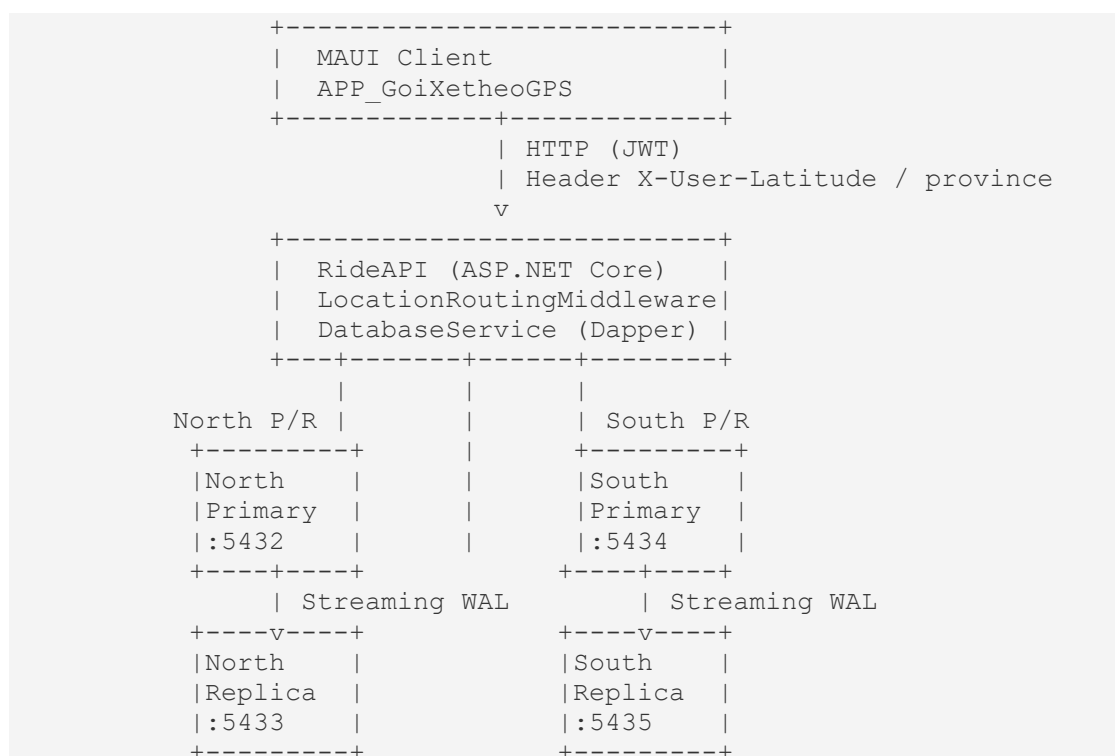
CHƯƠNG 2: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

2.1 Kiến trúc tổng thể

Hệ thống được xây dựng theo mô hình phân tán 4 tầng:

Tầng	Thành phần	Vai trò
Tầng 1 – Client	.NET MAUI (APP_GoiXetheoGPS)	Giao diện người dùng; tự động gửi vĩ độ X-User-Latitude và JWT Bearer khi gọi API
Tầng 2 – Query Router	ASP.NET Core (RideAPI)	LocationRoutingMiddleware xác định vùng; DatabaseService điều phối kết nối; Swagger, Admin Razor
Tầng 3 – Primary DB	PostgreSQL 16 (North & South Primary)	Nhận mọi thao tác đọc/ghi; phát WAL liên tục sang Standby
Tầng 4 – Replica DB	PostgreSQL 16 (North & South Standby)	hot_standby=on; chỉ đọc; phục vụ failover khi Primary ngừng hoạt động

Sơ đồ kiến trúc tổng thể:



2.2 Thiết kế cơ sở dữ liệu

2.2.1 Các bảng chính

Mỗi cụm (North và South) chứa cùng schema, khởi tạo từ init-north.sql / init-south.sql:

Bảng	Mô tả
Regions	Vùng địa lý (1 = North, 2 = South)
Customers	Thông tin khách hàng (liên kết Users)
Drivers	Thông tin tài xế và trạng thái hoạt động
Users	Tài khoản đăng nhập (Role: Admin / Customer / Driver)
AuthRefreshTokens	Refresh Token phía server
RevokedJwtTokens	Danh sách JWT đã thu hồi (blacklist)
Vehicles	Phương tiện gắn với tài xế
Trips	Chuyến đi (trạng thái, giá, tọa độ đón/trả, timestamps)
TripLocations	Các điểm/đoạn đường theo từng chuyến
Payments	Thanh toán theo chuyến
Reviews	Đánh giá sau chuyến
Promotions	Mã khuyến mãi
RideRequests	Yêu cầu chuyến (tọa độ đón/trả, trạng thái khớp lệnh)
DriverLocations	Vị trí GPS tài xế (cập nhật realtime)

2.2.2 Chiến lược phân vùng dữ liệu

– Dữ liệu được phân chia theo vùng địa lý. Mỗi node chỉ lưu dữ liệu của vùng tương ứng, đảm bảo cô lập và giảm độ trễ:

- Node North (Primary :5432 / Replica :5433): Dữ liệu khách hàng, tài xế, chuyến đi tại Hà Nội và các tỉnh Miền Bắc.
- Node South (Primary :5434 / Replica :5435): Dữ liệu tại TP.HCM và các tỉnh Miền Nam.

– Quy tắc định tuyến trong LocationRoutingMiddleware (thứ tự ưu tiên):

Ưu tiên	Nguồn xác định vùng	Chi tiết
---------	---------------------	----------

1	Header X-User-Latitude	Parse vĩ độ; nếu $\geq 16 \rightarrow$ NORTH, $< 16 \rightarrow$ SOUTH
2	Query string: province hoặc lat	Tra danh sách tỉnh/thành chuẩn hóa
3	Body JSON/Form: province hoặc lat	Áp dụng cho POST/PUT application/json
4	JWT Claims (regionId)	Một số controller ưu tiên regionId trong token
5	Mặc định	Không xác định được $\rightarrow$ SOUTH

2.3 Thiết kế Query Router (DatabaseService)

– DatabaseService là thành phần cốt lõi thực hiện:

- Nhận vùng (region) và loại thao tác (isWrite) từ controller.
- Nếu isWrite = false (đọc): Thử Primary trước; nếu thất bại thì thử Replica; nếu cả hai thất bại $\rightarrow$ ném InvalidOperationException "ALL_DB_NODES_DOWN".
- Nếu isWrite = true (ghi): Chỉ thử Primary; nếu không kết nối được $\rightarrow$ ném "MASTER_DOWN_CANNOT_WRITE" (không tự chuyển ghi sang Replica).
- DbRetryService bọc Polly: retry tự động khi gặp lỗi transient Npgsql hoặc timeout.

```

public async Task<NpgsqlConnection> GetConnectionAsync(string
region, bool isWrite)
{
    var (primaryConn, replicaConn) = GetRegionConnections(region);

    if (isWrite)
    {
        var primary = await TryOpenAsync(primaryConn);
        if (primary is not null)
            return primary;

        throw new
InvalidOperationException("MASTER_DOWN_CANNOT_WRITE");
    }

    var master = await TryOpenAsync(primaryConn);
    if (master is not null)
        return master;
}

```

```

var replica = await TryOpenAsync(replicaConn);
if (replica is not null)
    return replica;

throw new InvalidOperationException("ALL_DB_NODES_DOWN");
}

```

2.4 Cơ chế Replication (PostgreSQL 16 Streaming)

Thành phần	Cấu hình chính	Chức năng
Primary	wal_level=replica, max_wal_senders=5, hot_standby=on	Nhận mọi đọc/ghi; phát WAL stream liên tục
Standby	hot_standby=on, pg_basebackup stream từ Primary	Chỉ đọc (hot standby); nhận và apply WAL
Kết nối replication	primary_conninfo trong postgresql.auto.conf	Standby duy trì kết nối replication bền vững
Khởi tạo	pg_basebackup --wal- method=stream --write- recovery-conf	Tạo snapshot data + tự sinh standby.signal

CHƯƠNG 3: TRIỂN KHAI HỆ THỐNG

3.1. Cấu hình môi trường Docker

Toàn bộ 4 node PostgreSQL 16 được đóng gói trong 2 file Docker Compose:

- docker-compose.north.yml – north-master (:5432), north-slave (:5433)
- docker-compose.south.yml – south-master (:5434), south-slave (:5435)

– **Cấu hình mẫu – docker-compose.north.yml:**

```
version: '3.8'
services:
  north-master:
    image: postgres:16
    container_name: north-master
    environment:
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: secret
      POSTGRES_DB: ride_sharing
    command: postgres -c wal_level=replica -c max_wal_senders=10 -c
wal_keep_size=64MB -c hot_standby=on -c listen_addresses='*'
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U admin -d ride_sharing"]
      interval: 5s
      timeout: 5s
      retries: 12
    ports:
      - "5432:5432"
    volumes:
      - north_master_data:/var/lib/postgresql/data
      - ./init-north.sql:/docker-entrypoint-initdb.d/init.sql
      - ./enable-replication.sh:/docker-entrypoint-initdb.d/enable-
replication.sh
    networks:
      - north-net
```

```

north-slave:
  image: postgres:16
  container_name: north-slave
  depends_on:
    north-master:
      condition: service_healthy
  environment:
    POSTGRES_USER: admin
    POSTGRES_PASSWORD: secret
  command: >
    bash -c "
      export PGPASSWORD=secret;
      until pg_isready -h north-master -p 5432 -U admin; do sleep 2; done;
      rm -rf /var/lib/postgresql/data/*;
      until pg_basebackup -h north-master -U admin -D
/var/lib/postgresql/data -P -R --wal-method=stream; do sleep 2; done;
      echo 'hot_standby = on' >> /var/lib/postgresql/data/postgresql.conf;
      chown -R postgres:postgres /var/lib/postgresql/data;
      exec docker-entrypoint.sh postgres
    "
  ports:
    - "5433:5432"
  volumes:
    - north_slave_data:/var/lib/postgresql/data
  networks:
    - north-net
volumes:
  north_master_data:
  north_slave_data:
networks:
  north-net:

```

– **Cấu hình mẫu – docker-compose.south.yml:**

```
version: '3.8'
services:
  south-master:
    image: postgres:16
    container_name: south-master
    environment:
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: secret
      POSTGRES_DB: ride_sharing
    command: postgres -c wal_level=replica -c max_wal_senders=10 -c
wal_keep_size=64MB -c hot_standby=on -c listen_addresses='*'
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U admin -d ride_sharing"]
      interval: 5s
      timeout: 5s
      retries: 12
    ports:
      - "5434:5432"
    volumes:
      - south_master_data:/var/lib/postgresql/data
      - ./init-south.sql:/docker-entrypoint-initdb.d/init.sql
      - ./enable-replication.sh:/docker-entrypoint-initdb.d/enable-
replication.sh
    networks:
      - south-net
  south-slave:
    image: postgres:16
    container_name: south-slave
    depends_on:
      south-master:
        condition: service_healthy
    environment:
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: secret
```

```

command: >
    bash -c "
    export PGPASSWORD=secret;
    until pg_isready -h south-master -p 5432 -U admin; do sleep 2; done;
    rm -rf /var/lib/postgresql/data/*;
    until pg_basebackup -h south-master -U admin -D
/var/lib/postgresql/data -P -R --wal-method=stream; do sleep 2; done;
    echo 'hot_standby = on' >> /var/lib/postgresql/data/postgresql.conf;
    chown -R postgres:postgres /var/lib/postgresql/data;
    exec docker-entrypoint.sh postgres
    "

ports:
  - "5435:5432"

volumes:
  - south_slave_data:/var/lib/postgresql/data

networks:
  - south-net

volumes:
  south_master_data:
  south_slave_data:

networks:
  south-net:

```

– Khởi chạy hệ thống:

- Thứ tự gọi ý: bật Docker cho hai cụm CSDL, sau đó chạy Web API. Cách đơn giản nhất là mở terminal ở thư mục gốc của repository (nơi vừa thấy APP_GoiXetheoGPS vừa thấy RideAPI). Khi đó lệnh `docker compose -f APP_GoiXetheoGPS/ride-booking-docker/...` trở đúng tới file compose, và `cd RideAPI/RideAPI` đưa vào đúng project mà `appsettings.json` đang cấu hình `Host=localhost` với các cổng 5432–5435 trùng với map port trong Docker.

```

docker compose -f APP_GoiXetheoGPS/ride-booking-docker/docker-
compose.north.yml up -d
docker compose -f APP_GoiXetheoGPS/ride-booking-docker/docker-
compose.south.yml up -d
cd RideAPI/RideAPI
dotnet run --launch-profile http

```

- Nếu bạn đang đứng sẵn trong thư mục APP_GoiXetheoGPS (ví dụ mở terminal từ IDE tại đó), dùng đường dẫn compose ngắn hơn và lùi một cấp rồi vào API:

```
docker compose -f ride-booking-docker/docker-compose.north.yml up -d
docker compose -f ride-booking-docker/docker-compose.south.yml up -d
cd ../RideAPI/RideAPI
dotnet run --launch-profile http
```

- Sau khi API chạy, trình duyệt hoặc công cụ test gọi `http://localhost:5136` (đúng profile `http` trong `launchSettings.json`). Ứng dụng MAUI trên Android Emulator mặc định trỏ tới `http://10.0.2.2:5136` — cấu hình này nằm trong `WebApiServerConfig.cs`.

3.2. Cấu hình Replication

3.2.1. Cài đặt Primary

– Với mỗi miền, dựng một container primary (north-master và south-master), dùng image PostgreSQL 16. Thay vì chỉnh sửa trực tiếp file cấu hình trên đĩa, các tham số quan trọng cho replication được truyền vào lúc khởi động qua một dòng command trong file `docker-compose.north.yml` và `docker-compose.south.yml`. Cách này tương đương với việc ghi các giá trị vào `postgresql.conf`, nhưng thuận tiện hơn cho môi trường container vì mỗi lần `compose up` là một bản cấu hình thống nhất, hạn chế sai lệch giữa máy trong nhóm.

– Cụ thể, primary được bật `wal_level=replica` để WAL phục vụ nhân bản; `max_wal_senders=10` dành chỗ cho các kết nối gửi WAL; `wal_keep_size=64MB` giúp standby có thời gian bắt kịp dòng WAL; `hot_standby=on` chuẩn bị cho việc standby phục vụ truy vấn đọc; `listen_addresses=*` cho phép các container trong cùng mạng Docker — đặc biệt là standby — truy cập tới primary qua tên dịch vụ. Không đặt thêm `synchronous_commit` trong compose mà để PostgreSQL dùng giá trị mặc định; luồng WAL vẫn được stream xuống standby.

– Về tài khoản và cơ sở dữ liệu, image chuẩn đọc biến môi trường để tạo user admin, mật khẩu secret và database `ride_sharing`. Schema cùng dữ liệu mẫu được nạp một lần khi khởi tạo volume nhờ các script `init-north.sql` và `init-south.sql`. Để standby sau này có thể gọi `pg_basebackup`, bổ sung quy tắc trong `pg_hba.conf` bằng script `enable-replication.sh`: script nối thêm vào cuối file hai dòng cho phép kết nối replication với user admin (thay cho user replicator trong nhiều tài liệu mẫu) và mở kết nối thông

thường cho môi trường phòng lab. Cách chọn user chung như vậy giúp đơn giản hóa quản lý mật khẩu trong đồ án, đồng thời vẫn đáp ứng yêu cầu xác thực bằng md5 như cấu hình điển hình.

- Khi map cổng ra máy host, primary miền Bắc dùng cổng 5432, miền Nam dùng 5434, trùng với chuỗi kết nối trong tệp cấu hình ứng dụng Web API, nhờ đó nhóm có thể kiểm tra ứng dụng và replication trên một máy cá nhân mà không phải đổi địa chỉ phức tạp.

3.2.2. *Khởi tạo Replica*

- Sau khi primary đã sẵn sàng — container được đánh dấu healthy nhờ pg_isready trong Docker Compose — container standby (north-slave, south-slave) mới thực thi chuỗi lệnh khởi tạo. Standby không tự “trở thành” replica chỉ bằng cấu hình rỗng; nó cần một bản sao lưu trữ cấp vật lý đầy đủ từ primary.

- Trong đề tài, toàn bộ quy trình đó được gói trong một khối bash trong command của service slave: đặt biến môi trường mật khẩu, chờ primary phản hồi, xóa dữ liệu cũ trong thư mục dữ liệu của container để tránh lẫn bản cũ, rồi lắp lại pg_basebackup cho tới khi thành công. Lệnh dùng tùy chọn -R (tương đương vai trò của --write-recovery-conf trên các phiên bản cũ hơn) kết hợp --wal-method=stream, nhờ đó vừa nhận bản sao nền vừa duy trì luồng WAL song song — và PostgreSQL tự tạo các tệp cần thiết để standby nhận diện trạng thái hot standby, chẳng hạn standby.signal cùng thông tin nối về primary trong postgresql.auto.conf, mà không cần lưu sẵn các tệp đó trong kho mã nguồn.

- Sau bước backup, ghi thêm một dòng hot_standby = on vào postgresql.conf trong thư mục dữ liệu, chỉnh quyền sở hữu cho tiến trình postgres, rồi chuyển sang entrypoint chính thức của image để container chạy ổn định như một instance standby. Miền Nam được xử lý tương tự, chỉ thay tên host primary và cổng map ra ngoài (standby Bắc 5433, standby Nam 5435), thống nhất với phần cấu hình phân tán ở tầng ứng dụng.

- Đoạn cấu hình minh họa cho miền Bắc (docker-compose.north.yml):

```
command: >
  bash -c "
    export PGPASSWORD=secret;
    until pg_isready -h north-master -p 5432 -U admin; do sleep 2; done;
    rm -rf /var/lib/postgresql/data/*;
```

```

until pg_basebackup -h north-master -U admin -D
/var/lib/postgresql/data -P -R --wal-method=stream; do sleep 2; done;
echo 'hot_standby = on' >> /var/lib/postgresql/data/postgresql.conf;
chown -R postgres:postgres /var/lib/postgresql/data;
exec docker-entrypoint.sh postgres
"

```

– Đoạn cấu hình minh họa cho miền Nam (docker-compose.south.yml):

```

command: >
  bash -c "
    export PGPASSWORD=secret;
    until pg_isready -h south-master -p 5432 -U admin; do sleep 2; done;
    rm -rf /var/lib/postgresql/data/*;
    until pg_basebackup -h south-master -U admin -D
/var/lib/postgresql/data -P -R --wal-method=stream; do sleep 2; done;
    echo 'hot_standby = on' >> /var/lib/postgresql/data/postgresql.conf;
    chown -R postgres:postgres /var/lib/postgresql/data;
    exec docker-entrypoint.sh postgres
  "

```

3.3. Triển khai LocationRoutingMiddleware

LocationRoutingMiddleware chạy trên mọi request tới /api/..., gán context.Items["Region"] trước khi request đến controller:

```

        if (string.IsNullOrEmpty(region) &&
(HttpMethods.IsPost(context.Request.Method) ||
HttpMethods.IsPut(context.Request.Method)))
        {
            var contentType = context.Request.ContentType;
            var isJson = contentType != null &&
contentType.Contains("application/json");
            var isForm = contentType != null &&
contentType.Contains("application/x-www-form-urlencoded");

            if (isJson || isForm)
            {
                context.Request.EnableBuffering();
                try
                {
                    if (isJson)
                    {

```

```

        var body = await
JsonSerializer.DeserializeAsync<Dictionary<string,
JsonElement>>>(context.Request.Body);
        if (body != null)
        {
            if (body.TryGetValue("province", out
var p) && p.ValueKind == JsonValueKind.String)
                region =
LocationRoutingService.ResolveRegionFromProvince(p.GetString());
            else if (body.TryGetValue("lat", out
var l) && l.TryGetDouble(out var lat))
                region =
LocationRoutingService.ResolveRegionFromLatitude(lat);
        }
    }
    else if (isForm)
    {
        var form = await context.ReadFormAsync();
        var province = form["province"].ToString();
        if (!string.IsNullOrEmpty(province))
            region =
LocationRoutingService.ResolveRegionFromProvince(province);
        else if (double.TryParse(form["lat"], out
var lat))
            region =
LocationRoutingService.ResolveRegionFromLatitude(lat);
    }
}
catch
{
}
finally
{
    context.Request.Body.Position = 0;
}
}
}

```

3.4. Luồng Failover và Chế độ Read-Only

- Luồng xử lý đầy đủ khi Primary Miền Nam ngừng hoạt động:
- Kịch bản: Client MAUI gọi GET /api/rides/history?province=ho_chi_minh

1. LocationRoutingMiddleware → region = "SOUTH"
 2. RidesController.GetHistory() gọi
DatabaseService.GetConnectionAsync("SOUTH", isWrite: false)
 3. DatabaseService thử kết nối South Primary (:5434) – timeout 3s
 4. OperationalError: Primary không phản hồi
 5. isWrite = false → thử South Replica (:5435)
 6. Kết nối Replica thành công → trả NpgsqlConnection (Read-Only)
 7. Dapper truy vấn SELECT và trả HTTP 200 + data
- Kịch bản ghi: POST /api/rides (đặt chuyến)
 1. DatabaseService thử South Primary → thất bại
 2. isWrite = true → throw
InvalidOperationException("MASTER_DOWN_CANNOT_WRITE")
 3. RidesController bắt exception → return HTTP 503
Body: { "message": "Hệ thống đang ở chế độ chỉ đọc, không thể đặt chuyến" }

3.5. Các API Endpoint chính

– Loại API:

Giá trị	Ý nghĩa
Đọc (GetConnectionAsync, isWrite: false)	Có fallback replica khi primary không mở được kết nối.
Ghi (GetConnectionAsync, isWrite: true)	Chỉ primary; primary sập → MASTER_DOWN_CANNOT_WRITE / lỗi tương đương.
Đọc (ExecuteWithRetry, isWrite: false)	Giống trên, qua DbRetryService + Polly.
Ghi (ExecuteWithRetry, isWrite: true)	Ghi qua primary + retry; một số controller bắt MASTER_DOWN_CANNOT_WRITE → HTTP 503.
Đọc / Ghi (GetConnection)	Chỉ lấy chuỗi primary của vùng (DatabaseService.GetConnection), không qua GetConnec

	tionAsync → không fallback replica; primary sập thường lỗi kết nối (503 kiểu Npgsql trong từng controller).
Hỗn hợp	Trong một request vừa đọc vừa ghi với các chế độ khác nhau (chi tiết ở cột Mô tả).
Không CSDL	Không mở kết nối PostgreSQL.

– Bảng 1 — REST JSON (/api/...)

Method	Endpoint	Loại	Mô tả
POST	/api/auth/login	Đọc (GetConnecti onAsync, isWrite: false)	Đăng nhập — SELECT user; hoạt động khi primary sập nếu replica còn (đọc được).
POST	/api/auth/register	Ghi (GetConnecti onAsync, isWrite: true)	Đăng ký — kiểm tra trùng + INSERT; cần primary.
POST	/api/auth/refresh	Hỗn hợp	RefreshTokenService.FindValidAsync → đọc (isWrite: false); ReadUserByIdAsync → đọc; CreateAsync / RevokeAsync refresh token → ghi (isWrite: true). Primary sập: phần đọc có thể qua replica, phần ghi token cần primary.
GET	/api/auth/session	Không CSDL	Chỉ đọc claims JWT, không query DB.
POST	/api/auth/logout	Ghi (GetConnecti onAsync, isWrite: true)	Thu hồi refresh / ghi RevokedJwtTokens — cần primary từng lần gọi service.
GET	/api/rides/history	Đọc (ExecuteWit hRetry, isWrite: false)	Lịch sử chuyến — SELECT; có fallback replica.
POST	/api/rides	Ghi (ExecuteWith Retry, isWrite: true)	Đặt chuyến — INSERT; bắt MASTER_DOWN_CANNOT_WRITE → 503 chỉ đọc.

Method	Endpoint	Loại	Mô tả
POST	/api/payments	Ghi (ExecuteWithRetry, isWrite: true)	Cập nhật thanh toán — UPDATE; 503 khi primary ghi không được.
POST	/api/ratings	Ghi (ExecuteWithRetry, isWrite: true)	Đánh giá + cập nhật điểm TB tài xế (hai khối đều isWrite: true); 503 khi primary ghi không được.
GET	/api/distributed-db/stats	Đọc (GetConnectionAsync, isWrite: false)	Thống kê (JWT). Cùng action với route alias /api/database/stats và /api/distributeddb/stats.
GET	/api/distributed-db/stats/primary	Đọc (GetConnectionAsync, isWrite: false)	Chỉ vùng NORTH trong code hiện tại. Alias /api/database/..., /api/distributeddb/....
GET	/api/distributed-db/stats/secondary	Đọc (GetConnectionAsync, isWrite: false)	Chỉ vùng SOUTH (và route .../replica).
GET	/api/distributed-db/stats/replica	Đọc (GetConnectionAsync, isWrite: false)	Trùng action với stats/secondary.
POST	/api/drivers/update-location	Ghi (GetConnection)	INSERT vị trí — chỉ primary, không fallback replica; lỗi kết nối → 503 (không dùng message MASTER_DOWN_CANNOT_WRITE).
GET	/api/drivers/nearby	Đọc (GetConnection)	SELECT tài xế gần — chỉ primary; không fallback replica.
POST	/api/trips/request	Ghi (ExecuteWithRetry, isWrite: true)	Tạo chuyến qua TripService; không catch MASTER_DOWN_CANNOT_WRITE trong controller → có thể 500 sau retry.
GET	/api/trips/{id}	Đọc (ExecuteWithRetry, isWrite: false)	Chi tiết chuyến — có fallback replica.

Method	Endpoint	Loại	Mô tả
POST	/api/trips/accept	Ghi (ExecuteWith Retry, isWrite: true)	Tài xế nhận chuyến — giống cảnh báo TripController / TripService ở trên.
POST	/api/trips/comple te	Ghi (ExecuteWith Retry, isWrite: true)	Hoàn tất chuyến — giống trên.

– Bảng 2 — Razor Admin (/admin/..., cookie AdminOnly)

Method	Endpoint	Loại	Mô tả
GET	/admin/login	Không CSDL	Trang form đăng nhập.
POST	/admin/login	Đọc (GetConnection)	Thử tìm admin trên North (lat=20), không có thì South (lat=10) — hai lần mở primary.
POST	/admin/logout	Không CSDL	Xóa cookie JWT.
GET	/admin/dashboard	Không CSDL	Dashboard (dữ liệu từ claims + cookie scope).
POST	/admin/dashboard/scope	Không CSDL	Lưu scope vào cookie.
GET	/admin/trips	Không CSDL	Trang view (không query SQL trực tiếp trong action).
GET	/admin/nearby-drivers	Không CSDL	Tương tự.
GET	/admin/unauthorized	Không CSDL	Trang trạng thái.
GET	/admin/status/{code}	Không CSDL	Trang trạng thái.
GET	/admin/users	Đọc (GetConnection)	Danh sách user theo scope.
GET	/admin/users/create	Không CSDL	Form tạo mới.
POST	/admin/users/create	Ghi (GetConnection)	INSERT user (+ customer nếu cần).
GET	/admin/users/edit/{id}	Đọc (GetConnection)	Tải user để sửa.
POST	/admin/users/edit/{id}	Ghi (GetConnection)	UPDATE user.
POST	/admin/users/delete/{id}	Ghi (GetConnection)	DELETE user.
POST	/admin/users/toggle-lock/{id}	Ghi (GetConnection)	UPDATE khóa/mở khóa.

CHƯƠNG 4: KIỂM THỬ

4.1. Chiến lược kiểm thử

- Xây dựng 4 nhóm test case, kiểm thử tay theo hướng dẫn:
- Nhóm A – Kiểm thử Định tuyến theo vị trí: Đảm bảo mọi request được định tuyến đúng node theo province/latitude/JWT.
- Nhóm B – Kiểm thử Replication: Đảm bảo dữ liệu ghi vào Primary được đồng bộ lên Replica.
- Nhóm C – Kiểm thử Failover & Read-Only: Đảm bảo hệ thống hoạt động đúng khi Primary sập.
- Nhóm D – Kiểm thử Tích hợp MAUI + API: Đảm bảo ứng dụng MAUI hoạt động xuyên suốt toàn luồng.

4.2. Test Case chi tiết

- Nhóm A – Định tuyến theo vị trí

STT	Tên test	Đầu vào	Kết quả mong đợi
A1	Định tuyến miền Bắc qua province (query)	GET API có path /api/...; query ?province=Ha%20Noi (hoặc chuỗi chuẩn hóa tương đương ha noi trong NorthLocations)	HttpContext.Items["Region"] = NORTH; các service dùng vùng này kết nối chuỗi North (5432/5433)
A2	Định tuyến miền Nam qua province (query)	Query ?province=Ho%20Chi%20Minh (hoặc tương đương ho chi minh / hcm trong SouthLocations)	Region = SOUTH
A3	Định tuyến miền Bắc qua header vĩ độ	Header X-User-Latitude: 21 ($\geq$ 16), không có province trùng South	Region = NORTH
A4	Định tuyến miền Nam qua header vĩ độ	Header X-User-Latitude: 10.8 ($<$ 16)	Region = SOUTH

A5	Định tuyến qua query lat	Không header vĩ độ; query ?lat=20.5	Region = NORTH
A6	Mặc định khi không có gợi ý vùng	Request /api/... không header vĩ độ, không province/lat query, không body liên quan	Region = SOUTH (mặc định middleware)
A7	JWT ưu tiên vùng (API có [Authorize])	Gọi POST /api/rides với JWT có regionId=1 nhưng header/query gợi Nam	Controller dùng NORTH theo ResolveRegionFromClaims (đúng mã hiện tại)

– Nhóm B – Replication

STT	Tên Test	Đầu vào	Kết quả mong đợi
B1	Đồng bộ INSERT	INSERT chuyển mới vào North Primary :5432	Bản ghi xuất hiện trên North Replica :5433 trong < 1s
B2	Đồng bộ UPDATE	UPDATE status chuyển trên South Primary	Replica South hiển thị status mới sau < 1s
B3	Replica là Read-Only	INSERT trực tiếp vào Replica :5433	PostgreSQL lỗi: cannot execute INSERT in a read-only transaction
B4	Đo độ trễ replication	10 INSERT liên tục, đo timestamp Primary vs Replica	Độ trễ trung bình < 200ms
B5	Kiểm tra pg_stat_replication	SELECT * FROM pg_stat_replication trên Primary	state = streaming, sent_lsn $\approx$ write_lsn $\approx$ flush_lsn

– Nhóm C – Failover & Chế độ Read-Only

STT	Tên Test	Đầu vào	Kết quả mong đợi
C1	Đọc khi South Primary sập	docker stop south-master; GET /api/rides/history (province=ho_chi_minh)	HTTP 200 + data từ South Replica :5435
C2	Ghi khi South Primary sập	docker stop south-master; POST /api/rides	HTTP 503 + "không thể đặt chuyến" (MASTER_DOWN_CANNOT_WRITE)
C3	Thanh toán khi Primary sập	docker stop south-master; POST /api/payments	HTTP 503 + thông báo chỉ đọc
C4	Phục hồi sau Primary online	docker start south-master; POST /api/rides	HTTP 200, chuyển được tạo thành công
C5	Sập South không ảnh hưởng North	docker stop south-master; request province=ha_noi	HTTP 200, North Primary hoạt động bình thường
C6	Cả 2 Primary cùng sập – đọc	Stop north-master & south-master; GET /api/distributed-db/stats	HTTP 200 từ cả 2 Replica với DTO fallback
C7	Cả 2 node sập (Primary + Replica)	Stop cả 4 container; gọi API đọc	Lỗi ALL_DB_NODES_DOWN, API trả 503/500
C8	Đăng nhập khi Primary sập	Stop south-master; POST /api/auth/login	HTTP 200 – login dùng isWrite:false, Replica phục vụ

– Nhóm D – Tích hợp MAUI + API

STT	Tên Test	Đầu vào	Kết quả mong đợi
D1	MAUI gửi đúng X-User-Latitude	App MAUI chạy trên Android Emulator (lat=10.8)	Header X-User-Latitude: 10.8 được gửi, region=SOUTH
D2	Xem lịch sử chuyển khi failover	Stop south-master; vào màn TripTrackingPage	Danh sách chuyển hiển thị từ Replica
D3	Màn hình bản đồ (Mapbox)	Nhập địa điểm trên HomeMapPage	Mapbox Geocoding API trả kết quả; UI hiển thị gợi ý
D4	Admin dashboard	Đăng nhập /admin với tài khoản Role=Admin	Dashboard quản lý user/driver hiển thị đúng
D5	Base URL mặc định	Chạy API cổng 5136; MAUI Android Emulator	WebApiServerConfig dùng http://10.0.2.2:5136; máy thật / iOS dùng localhost:5136
D6	Header khi gọi API	ApiClient / TripDataStore có token	Gửi Authorization: Bearer ... và X-User-Latitude khi có GPS
D7	Đăng nhập từ app	Nhập email/mật khẩu seed; có vĩ độ / tỉnh nếu form hỗ trợ	Nhận JWT, vào được luồng chính
D8	Thống kê DB phân tán từ app	Màn hình / service gọi DistributedDatabaseService	Hiển thị 2 vùng hoặc fallback endpoint theo mã

		tới GET /api/distributed-db/stats (JWT)	
D9	Đặt chuyến (happy path)	Khách đặt chuyến qua API với JWT + tọa độ hợp lệ	POST /api/rides hoặc luồng TripDataStore trả thành công khi DB khỏe

CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Hạn chế hiện tại

- Chưa triển khai automatic promotion: Standby không tự lên làm Primary khi Primary sập hoàn toàn – cần công cụ như Patroni, repmgr, hoặc Keepalived.
- Chưa xử lý split-brain trong trường hợp network partition (cả hai node tin mình là Primary).
- Định tuyến theo vĩ độ ≥ 16 là mô hình đơn giản, chưa phản ánh đầy đủ ranh giới hành chính (ví dụ: Nghệ An, Hà Tĩnh ở ranh giới Bắc–Trung).
- GET /api/distributed-db/stats/primary chỉ thống kê North Primary; endpoint secondary chỉ thống kê South Replica – chưa tổng hợp cả 4 node.
- POST /api/auth/register chưa hash mật khẩu ở nhánh INSERT (chỉ VerifyPassword dùng BCrypt khi đăng nhập).
- Dữ liệu người dùng không di chuyển được giữa vùng – chưa hỗ trợ user chuyển vùng sinh sống.

5.2. Hướng phát triển

- Tích hợp Patroni hoặc repmgr để quản lý High Availability tự động (auto-failover + promote) cho PostgreSQL.
- Triển khai trên Kubernetes với StatefulSet và PersistentVolumeClaim để tận dụng auto-recovery và orchestration.
- Thêm vùng thứ 3 (Miền Trung) và áp dụng consistent hashing để phân vùng linh hoạt hơn.
- Cài đặt monitoring và alerting với Prometheus + Grafana để phát hiện và cảnh báo sự cố node sớm.
- Nghiên cứu áp dụng CockroachDB hoặc YugabyteDB (NewSQL phân tán) như là phương án thay thế PostgreSQL đơn lẻ.
- Thêm connection pooling (PgBouncer) để tối ưu hiệu năng khi số lượng kết nối đồng thời tăng cao.

- Hoàn thiện tính năng Promote Standby: khi Primary sập lâu, tự động promote Standby lên Primary và thông báo Admin.